

# DL066 精度累积与重计算实验

这次实验的目标是把三种节省策略分开验证：数值精度改变是否可接受、微批梯度是否对应同一目标、激活重算是否复现原函数。一个方法能运行、能下降，并不足以证明它实现正确。

## 1 环境与验证范围

推荐独立Python3.11或3.12，按requirements.txt安装。当前结果实测Python3.12.14、PyTorch2.14.1+cpu，CPU单线程，无GPU。CPU BF16 autocast实际执行；CUDA FP16与GradScaler没有执行，结果里明确标为not\_run。若你的CPU/PyTorch组合不支持所用BF16算子，应记录具体错误并先完成FP32、累积和重算，不要伪造支持结果。

```
python -m pip install -r requirements.txt
python -m unittest -v test_experiment.py
python -O -m unittest -v test_experiment.py
python experiment.py --output outputs
python make_figures.py
```

命令在单元目录运行，默认重建同名文件。扩展实验用--output另存。网络与数据都是本课小规模原创，无外部数据、账户或预训练权重。保存的12个pt文件是真实训练的最终模型，加载只使用可信来源与weights\_only=True。

## 2 先观察浮点数

执行下面的小实验，预测每次转换后会出现什么。不要把torch.finfo.tiny误读成最小次正规数。

```
import torch
v = torch.tensor([1e-8, 1., 1. + 2** -10, 70000.])
for dtype in [torch.float32, torch.float16, torch.bfloat16]:
    print(dtype, torch.finfo(dtype))
    print(v.to(dtype).float())
```

FP16会丢失第一个数并让最后一个数溢出，BF16可以覆盖这两个数量级，却分不清中间两个接近1的数。解释“范围”与“局部刻度”分别由什么决定。再比较先缩放后转换、先转换后缩放两条路径，并写出为什么不能从0恢复原梯度。

## 3 单步核对基线

Net包含四个Linear(64,64)加Tanh，最后Linear(64,1)。主要对照没有BatchNorm或Dropout。固定同一初始化和同一批数据，计算FP32全批MSE及梯度向量。读取comparisons，先确认fp32对自身误差为0，然后比较BF16、正确累积、重算和错误等权累积。

```
import copy
import experiment as e
x, y = e.data()
base = e.model()
loss, reference = e.gradients(base, x, y)
_, accumulated = e.gradients(copy.deepcopy(base), x, y, sizes=[31,47,50])
print(e.relative_error(reference, accumulated))
```

正确累积相对误差应接近浮点舍入量级，不能要求跨平台逐位一致。BF16误差不为0并不意外。本次约0.00238，应结合完整训练而不是只给一个任意阈值就宣称普适安全。

14项测试普通与-O都应通过。检查其中不是只测试“函数返回了值”，而是含正确与故意错误分支的对照。例如错误权重、未保留Dropout随机状态、BatchNorm微批差异都必须被实际触发。

## 4 用不等长微批练习归约

纸笔先推导31、47、50三个微批的权重：它们分别占128个样本的31/128、47/128、50/128。

gradients只在开头清一次梯度，每个微批前向和反向时都不更新参数，最后train调用一次step。

把wrong\_equal=True传入，只改变loss权重，不改变微批分割。观察相对误差由约 $1.27\text{e-}7$ 增大到约0.05168。这个对照把“分母错误”与“微批导致的浮点舍入”区分开来。

**延伸。** 假如每个样本是长度不同的序列，loss按有效token平均，权重应该使用有效token数量。如果用样本数加权，你优化的是另一个目标。先用两条不同长度的序列手算，再写代码。

## 5 看重算保存了什么

saved\_payload用autograd hooks记录前向为反向保存的张量，排除参数storage，并按底层存储去重。运行后完整路径164864字节、重算66560字节。根据函数的scope文字，写一句准确描述，不得用“峰值显存”替换字段名称。

查看Net.forward: checkpoint只包住body，显式use\_reentrant=False。然后看timed\_backward，计时在没有hooks的独立运行中完成，包含清梯度、前向和反向，没有优化器更新。warmup后7组，每组5次，保存原始块值。计时变化正常，不把重算速度设置为测试必须满足的固定倍数。

## 6 两个随机或跨样本反例

第一组给模型加Dropout。固定同一随机种子，比较非重算与preserve\_rng=True重算；本次梯度一致。再次固定种子，但使用preserve\_rng=False，观察相对误差约0.5413。解释同一批输入为什么仍不能保证同一函数。

第二组给模型加BatchNorm。比较全批与31、47、50微批正确加权；误差约0.1986。loss分母正确并没有保住前向函数，因为BatchNorm每次使用的批统计不同。不要为了让测试“绿灯”而放宽容差到容纳这个差异；它应该作为有解释的反例被保留。

## 7 实际训练与checkpoint复核

每个初始化种子11、23、37运行四种模式，每种160次SGD更新。训练集128例，独立测试256例。所有模型最后统一用FP32推理报告测试MSE，使评价路径一致。

```
import numpy as np
import torch
import experiment as e
z = torch.load('outputs/bf16_seed11.pt', weights_only=True)
m = e.model(z['seed'])
m.load_state_dict(z['state_dict'])
a = np.load('outputs/data.npz')
with torch.no_grad():
    value = torch.nn.functional.mse_loss(
        m(torch.from_numpy(a['test_x'])), torch.from_numpy(a['test_y']))
print(float(value))
```

种子11 BF16结果约0.124405。请为全部12个文件做同样复核，并比较JSON终点。这些文件包含最终权重和配置，但没有完整优化器/RNG状态，因此用于推理复核，不能冒称严格中途续训包。

## 8 Notebook与文档

Notebook从新进程按顺序执行，重训全部12组，输出到notebook\_outputs。数值演示、测试和图均保留，图片通过PNG数据内嵌，不只是指向本地路径的Markdown。顺序执行不等于检验过浏览器或外进程内核传输。

```
python create_notebook.py
python execute_notebook.py experiment.ipynb
python build_pdf.py lecture.md
python build_pdf.py lab.md
python build_pdf.py answers.md
```

## 9 练习与提交

- 列出FP16与BF16的指数位、小数位，解释1附近间距与可表示范围的区别。
- 两个微批有效元素数为3和9，平均梯度向量分别为(2,-1)和(-2,3)。求正确平均梯度与错误等权结果。
- 一条25层链每层保存A，按每k层一段重算。用简化模型寻找合适k，指出被忽略的内存对象。
- 写出一次累积中每个微批都清梯度会发生什么；再解释每个微批都step的另一种错误。
- 为CUDA FP16加GradScaler写一份实验计划：说明unscale、梯度裁剪和update发生在哪个时点，并列出现当前CPU结果无法支持的声明。
- 解释为什么本课重算减少59.63%的hooks保存载荷，不能被表述为总训练内存下降59.63%。

提交单步误差表、三种子终点、全部原始时间块、保存量定义、两个失败反例及一段有范围的结论。答案在下一份文件中，先自行推导。